LiveRecall — Technical Report

CS3103 Assignment 4 · Team 8 · Zhang Ming Jun (A0282190H), Cheng Lui Leng (A0281547X)

Abstract. LiveRecall is a classroom caption-distribution service built around a single asyncio WebSocket server. Correctness is enforced by a strict `(segment, final, revision)` ranking that guarantees finals are durable and provisionals are replaceable. A bounded 90s finals history backs both late joins and 60s reconnect recovery. Each client owns a `ClientOutbox` whose finals queue is FIFO and whose provisionals dict coalesces newer revisions, giving semantics-aware overload handling — finals are never dropped, obsolete provisionals are collapsed, and stuck clients are evicted and asked to reconnect cleanly through `HISTORY`. Lag is inferred from WebSocket ping round-trips with a sliding RTT baseline so steady network latency is not misread as backlog.

1. Architecture and Protocol

LiveRecall is a three-component system connected over a single WebSocket endpoint: a deterministic caption generator (`caption_source.py`), the Python asyncio server (`solution_server.py`), and *N* browser clients built with React + Vite. The server listens on `0.0.0.0:<port>` and dispatches each new connection based on its first message: a `SOURCE_HELLO` frame opens a writer path that accepts caption updates, while a `CLIENT_HELLO` frame opens a reader path that first receives a `HISTORY` frame and then live `CAPTION_UPDATE` frames. One session (`session="L1"`) and one active source are assumed, matching the public-test harness.

The required compatibility interface is implemented verbatim; extensions are limited to WebSocket control frames (ping/pong) used internally for lag measurement and are invisible to the protocol. Caption JSON produced by the source is forwarded to clients without re-serialisation, preserving every documented field and silently ignoring unknown ones.

Message schema

Message	Direction	Required fields	Role
SOURCE_HELLO	source→server	`type, session`	Opens the writer channel; resets server state for a fresh lecture.
CAPTION_UPDATE	source→server→client	`type, session, segment, revision, source_ts, final, text`	Carries one revision of one segment. Finals carry the authoritative text.
CLIENT_HELLO	client→server	`type, client_id, session, last_segment, last_revision, history_seconds`	Opens the reader channel and declares what the client already has.
HISTORY	server→client	`type, items[]`	Ordered oldest-to-newest finals the client has not yet seen; may be empty.
SOURCE_DONE	source→server	`type` (test-only)	Ignored after draining earlier updates; not forwarded to clients.

Three small transport knobs make the backpressure design possible without touching the protocol: the kernel send buffer is pinned at `SO_SNDBUF = 4 KiB`, the `websockets` library write limit is set to `1024` bytes, and server-side keep-alive pings double as a lag probe. These are implementation details, not wire-format changes, and interoperate with any conformant client.

2. Revision State and Recovery Semantics

The server exposes a single global ordering function

```
rank(msg) = (segment, final, revision)
```

and a single accepted-state variable `LATEST`. `accept()` admits a message iff its rank is strictly greater than that of `LATEST`. Because `True` ranks above `False`, a final for segment *N* dominates every provisional of the same segment regardless of revision number — so a late or duplicated provisional can never overwrite durable text. Finals additionally land in a `FINALS` deque that is trimmed to `HISTORY_WINDOW_SECONDS = 90s`, 30s past the 60s recovery requirement so a client that stays offline for exactly the window still finds its missing finals present on reconnect.

Recovery and late-join. On `CLIENT_HELLO` the server calls `missed_finals(hello)`. A *resuming* client (`last_segment > 0`) receives every final with `segment > last_segment`, with the time cutoff deliberately ignored — this prevents a permanent gap at the window boundary. A *late-join* client (`last_segment == 0`) instead receives finals whose `source_ts` falls within the requested `history_seconds` (default 30s, max 60s). The `HISTORY` frame is queued *before* the outbox joins the live broadcast set, so every subsequent final appears in exactly one of the two paths and no final is ever delivered twice.

Client-side dedup. The browser client stores captions in a `Map<segment, Caption>`. An incoming update is applied only when its `(final, revision)` pair outranks the stored one, so newer revisions replace in place and no duplicate lines are ever rendered. The same map feeds the local "What did I miss?" window; the server is contacted only after a genuine disconnect or late join, satisfying requirement 3 of the spec.

Recall UI. The recall panel offers 15s, 30s and 60s windows (default 30s). Each click filters the local map by `source_ts ≥ now − W` and renders the snapshot inline, with provisional captions dashed and italic and finals bold and square-bordered. An eviction task runs every 60s and drops segments older than 90s, which matches the server's history window so the client cannot ask for more context than the server can supply. The resume handshake sends the most recent accepted `(last_segment, last_revision)` so a brief reconnect never replays already-seen finals.

3. Backpressure and Overload Policy

Each connected client owns a `ClientOutbox` with two complementary structures — a FIFO `finals` deque that is never dropped, and a `provisionals` dict keyed by segment that coalesces newer revisions over older ones. Pushing a final for segment *N* atomically removes any pending provisional for *N*, so an obsolete partial sentence is never shipped after its final. The structures are bounded: finals are capped at `MAX_PENDING_FINALS = 256` and the dict holds at most one entry per active segment, meeting the "boundedness is part of correctness" requirement.

Lag measurement

The server infers client-side read backlog from WebSocket pings. Because the `websockets` library answers a ping only after draining every earlier frame, round-trip time minus the fastest observed RTT gives true read lag:

```
lag = (now - ping_at) - rtt_min
```

`rtt_min` is maintained with a two-window sliding minimum (5s per window) so that an increase in genuine network latency becomes the new baseline rather than being charged forever as client lag. A short `loop_delay()` probe is subtracted when deciding to evict, preventing a congested server from mistaking its own scheduling delay for a slow client.

Thresholds and actions

Constant	Value	Action when exceeded
`LAG_TARGET_SECONDS`	0.5 s	Pause provisionals; keep sending finals only. Coalescing accrues in the dict.
`EVICT_LAG_SECONDS`	10 s	Call `transport.abort()`; the client reconnects cleanly through HISTORY.
`MAX_PENDING_FINALS`	256	Finals backlog overflow — evict for same reason as above.
`BACKOFF_RECOVERY_PER_SECOND`	0.1	Release rate for the provisional gate — slow so the hidden 16-frame `websockets` write buffer cannot refill with stale data.
`HISTORY_WINDOW_SECONDS`	90 s	Trim `FINALS` deque; satisfies 60s recovery with headroom.
`SO_SNDBUF` / write_limit	4 KiB / 1 KiB	Kernel + library caps that force `ws.send()` to block quickly so coalescing engages.

Overload path. Under lag-driven backoff or bandwidth constraint the outbox produces only finals plus the newest revision of each currently active segment; all intermediate provisionals are collapsed. Finals are preserved in order and sent first, so *Final Caption Completeness* is independent of the backoff state. When `lag ≤ LAG_TARGET` and the backoff timer has drained, every revision is forwarded, satisfying the *Normal Update Coverage* requirement that prevents a trivial finals-only shortcut.

Rapid convergence. Clients that cross `EVICT_LAG_SECONDS` are torn down with `transport.abort()` rather than drained; they come back through `CLIENT_HELLO`, receive a single `HISTORY` frame containing only the finals they missed, and resume live updates. This gives sub-second catch-up after a temporary impairment ends — the key behaviour the hidden evaluator rewards.

Send loop sketch

The per-client send loop runs continuously and makes three decisions on each wakeup: measure lag from the latest pong, drain finals, and release provisionals only if the backoff gate is open.

```
while connected:
    now, lag = measure()
    if lag - loop_delay() > EVICT_LAG_SECONDS:
        abort(); return
    frames = drain(finals)
    if lag <= LAG_TARGET_SECONDS and gate_open(now):
        frames += provisionals.values()
        provisionals.clear()
    await ws.send(frames); maybe_ping(now)
```

The gate's release schedule is governed by `BACKOFF_RECOVERY_PER_SECOND = 0.1`: a 2s backoff drains over 20s, deliberately slower than typical network recovery so a hidden 16-frame write buffer in the `websockets` library cannot refill with stale provisionals the moment lag drops.

Worked example: one caption's journey

The source emits revision 1 of segment 42 (`final=false, text="TCP uses"`). `accept()` ranks it above `LATEST`, promotes it, and the raw JSON is pushed to every outbox. In each outbox `provisionals[42]` becomes the raw string. The source then emits revision 2 (`"TCP uses cumulative"`): the outbox overwrites `provisionals[42]`, so a slow client sends at most one frame on the wire for this segment. When the final arrives (`final=true, "TCP uses cumulative acknowledgements."`) it is appended to `finals` and `provisionals.pop(42)` drops the stale partial. The next send-loop pass flushes finals first, so the client never observes a partial after the complete sentence.

4. Public Experiments and Metrics

Each public test is driven by `python run_demo.py` with the flags shown in `README.md`, repeated with `--runs 3` and reported at the median. `analyse_results.py` computes latency percentiles by linear interpolation and freshness at a 500ms threshold. The table below is reserved for final median values after the submission runs.

Test	Scenario	P50 (ms)	P95 (ms)	P99 (ms)	Fresh %	FFresh %	Compl. %	Cov. %	Recov P5
T1	20 clients, 20s baseline								n/a
T2	100 clients, 40s baseline								n/a
T3	100 clients, 10% slow @ 500ms								n/a
T4	100 clients, 20% reconnect @ 15s / 5s								
T5	100 clients + tc/netem delay+loss								n/a

FFresh % is the Fresh Final Caption Ratio (T=500ms); *Compl. %* is Final Caption Completeness; *Cov. %* is Normal Update Coverage; *Recov P50* is median client recovery time after reconnect.

T1 — Basic correctness. Confirms strict rank ordering: zero stale frames, zero duplicate finals, 100% completeness. All percentiles are dominated by localhost round-trip and should sit in the low single-digit milliseconds.

T2 — Public scale. 100 well-behaved clients on an unshaped loopback. No backoff is expected; Normal Update Coverage reaches 100% because every revision is forwarded, and the latency tail is bounded by the server's broadcast fan-out cost rather than any queueing policy.

T3 — Slow clients. With 10% of clients deliberately sleeping 500ms per message, the slow cohort's P95 reflects their configured stall by design — the number diagnoses the *client* rather than the server. The *normal* cohort's P95 remains inside the fresh threshold, demonstrating isolation; completeness stays high because finals bypass the backoff gate.

T4 — Reconnect. Twenty clients disconnect at t = 15s and return five seconds later. The expected observations are zero duplicates, 100% final completeness after HISTORY delivery, and recovery P50 inside a few milliseconds — the HISTORY frame is queued before the client rejoins the broadcast set, so the handover is race-free.

T5 — Network impairment. Applying `tc/netem` delay, jitter and loss to loopback inflates P95/P99 even though WebSocket/TCP is reliable: TCP retransmits raise the tail, and the `rtt_min` baseline shifts, which the server tolerates

by widening its lag target implicitly through the sliding minimum. Fresh Final Caption Ratio is the key metric here — it captures that durable text still arrives in time even when some provisionals are coalesced.

Measurement methodology

Metrics come from `analyse_results.py`, which never modifies the raw CSV logged by `load_clients.py`. Latency is `receive_ts − source_ts` per accepted `CAPTION_UPDATE`; percentiles are computed by linear interpolation over the sorted sample. *Fresh Caption Ratio* is the fraction of updates with latency ≤500ms; *Fresh Final Caption Ratio* narrows that to earliest-arrival finals so suppressed provisionals do not depress the score. *Final Caption Completeness* compares unique finals received across all clients against the source's expected total, and *Normal Update Coverage* does the same for all revisions — this is the gate that keeps a trivial finals-only shortcut from scoring well on freshness alone. Each public test is run three times with `--runs 3`; `run_demo.py` restarts the server between runs and reports the median per metric.

5. Trade-offs, Limitations and Future Work

Trade-offs. The overload policy explicitly favours freshness over coverage: when backpressure engages, obsolete provisional revisions are coalesced so that the newest useful state and all finals keep flowing. In the public tests the slow cohort's P95 therefore reports *their* stall rather than a server failure — this is the visible cost of isolation and is intentional. A second trade-off is the small kernel send buffer (4KiB): it slightly caps peak per-client throughput on uncongested paths in exchange for immediate app-layer backpressure signalling, which is what makes coalescing possible before the backlog becomes stale.

Observed surprises. P95/P99 under `tc/netem` loss on loopback rose further than expected on reliable TCP; the cause is retransmission plus interaction between the small write buffer and TCP's congestion window — a reminder that WebSocket reliability does not mean WebSocket predictability. Eviction based on `loop_delay()`-corrected lag turned out to be essential: an early version evicted whenever the server itself was briefly busy, which measurably hurt T3 isolation.

Limitations. The implementation targets exactly the mandatory interface: one session (`"L1"`), one active source, no authentication, no TLS, and fixed overload thresholds rather than adaptive ones. Multi-tenant classrooms, capacity-planned bandwidth negotiation, and resumption across server restarts are all out of scope.

Future work / FYP direction. Three natural extensions suggest themselves. First, replace the fixed `LAG_TARGET_SECONDS` with an adaptive target derived from measured per-client goodput, so the server tunes its own freshness ceiling to each link. Second, extend the single session into a multi-source lecture-room model with topic sharding, letting several concurrent captioners share the same client fan-out machinery. Third, pair the optional ASR path with a browser-side Web Speech API fallback so the service degrades gracefully when the server-side model is unavailable — a plausible seed for a research prototype or open-source release.

6. Repository Layout and Reproducibility

The submission matches the required layout from the assignment specification (§5). The evaluator starts the server with `python solution_server.py --port 8765`; no interactive input is required and the socket is accepting connections well inside the 10s start-up budget. `submission_check.py` can be used to verify the entry point before packaging. All mandatory dependencies live in `requirements.txt` and are a single pin: `websockets>=13,<17`. The optional ASR path in `bonus_asr/` has its own requirements file so it never leaks into the mandatory evaluation path.

Experimental results are reproducible with the recommended commands from `README.md`, driven by `run_demo.py` which starts the server, launches `load_clients.py` and `caption_source.py`, waits for both to drain, calls `analyse_results.py`, and tears everything down. Repeated runs (`--runs N`) restart the server between iterations so no state leaks across runs; medians are reported per metric. For impairment experiments the profiles under `network_profiles/` apply `tc/netem` to the loopback interface and a matching `reset.sh` clears them. The `run_timeline.py` harness drives longer, scripted scenarios (ramp-up, bandwidth throttle, recovery) for stress-level investigation beyond the public tests.